

Projet Big Data – Rapport Final



LIMAM Mohamed Limam
Yanis Zidi
Alexandre Bouharira
Ilyes OubBea
Samuel Silva

Sommaire

1. Introduction.....	2
2. Architecture globale du projet.....	3
3. Développement technique.....	5
3.1 Mise en place de l'environnement.....	5
3.3 Exercice 4 - Dashboard (screenshots).....	6
3.2 Exercice 5 – Pipeline de Machine Learning.....	8
Objectif.....	8
Chargement des données.....	9
Prétraitement.....	9
Entraînement du modèle.....	11
Tests et qualité du code.....	12
4. Répartition des tâches.....	12
5. Difficultés rencontrées.....	13
6. Conclusion.....	13

1. Introduction

Dans le cadre du module Big Data, nous avons réalisé un projet visant à concevoir une architecture complète de traitement de données à grande échelle. L'objectif était de mettre en place un pipeline structuré permettant de charger, traiter, analyser et exploiter des données issues d'un jeu de données réel.

Nous avons travaillé à partir des données publiques des trajets de taxis new-yorkais (Yellow Taxi Trip Data). Ce jeu de données, volumineux et structuré, constitue un cas d'étude pertinent pour expérimenter les problématiques liées au stockage, au traitement distribué et à l'apprentissage automatique.

Le projet s'inscrit dans une démarche progressive : chaque exercice permettait d'ajouter une nouvelle brique technique à l'architecture globale. L'objectif final était d'aboutir à un pipeline cohérent, reproductible et automatisable.

2. Architecture globale du projet

L'architecture repose sur plusieurs composants complémentaires :

- **Stockage objet : MinIO** (compatible S3)
- **Traitement et Machine Learning : Python (pandas, scikit-learn)**
- **Conteneurisation : Docker**

Le pipeline général suit les étapes suivantes :

1. Chargement d'un fichier Parquet depuis MinIO
2. Prétraitement et nettoyage des données
3. Entraînement d'un modèle de régression
4. Évaluation des performances
5. Génération de prédictions

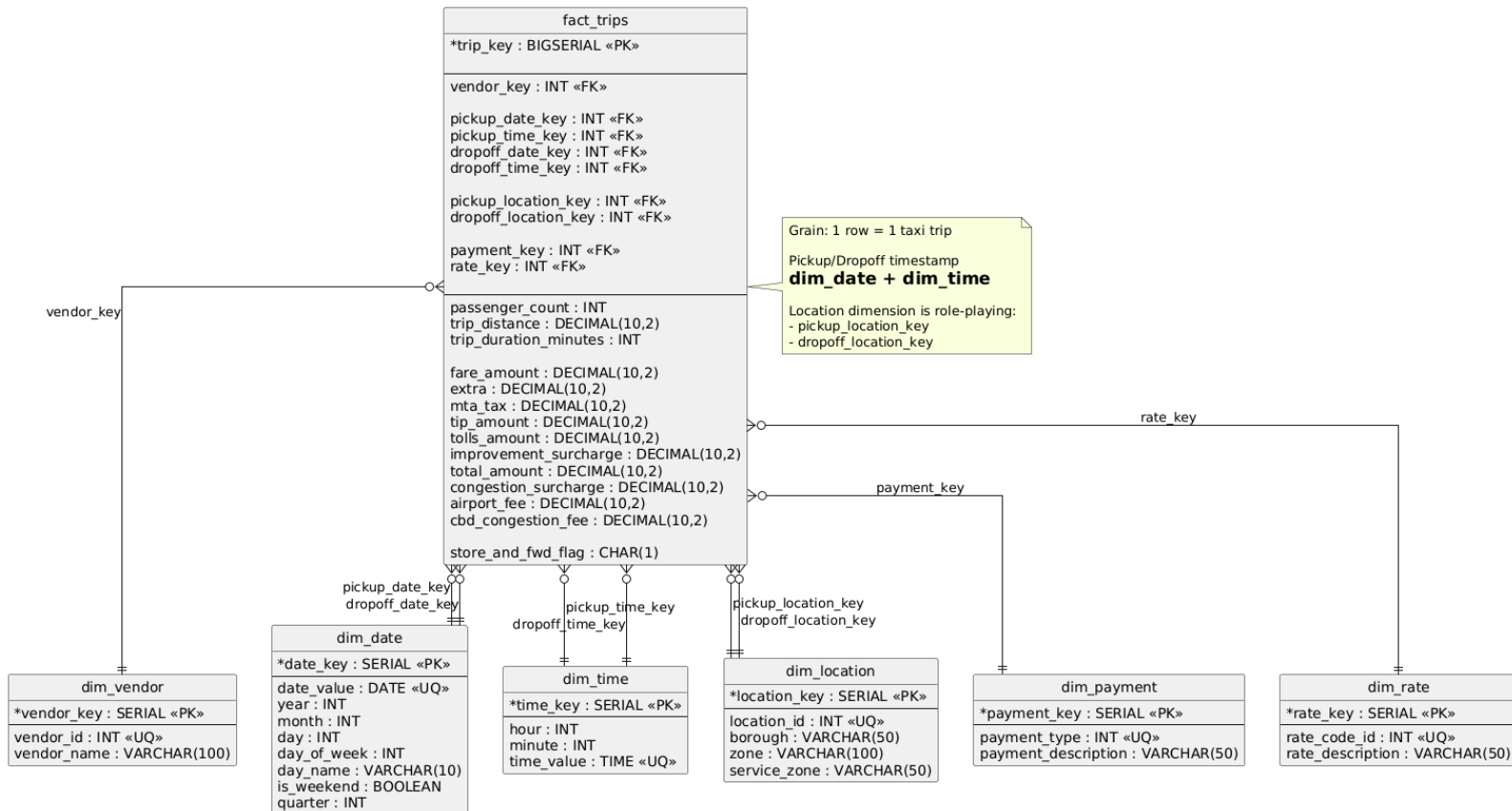
Cette architecture sépare clairement les responsabilités et permet une meilleure maintenabilité. Chaque composant peut évoluer indépendamment des autres.

```

├── data
│   ├── external
│   ├── interim
│   ├── processed
│   └── raw
├── Docker
│   ├── Dockerfile
│   ├── start-spark.sh
│   └── docker-compose.yml
├── ex01_data_retrieval
│   ├── build.sbt
│   ├── data
│   │   └── raw
│   │       ├── yellow_tripdata_2024-01.parquet
│   │       └── yellow_tripdata_2025-11.parquet
│   └── src
│       ├── main
│       │   ├── resources
│       │   └── scala
│       │       └── Main.scala
│       └── test
│           ├── resources
│           └── scala
├── ex02_data_ingestion
│   ├── build.sbt
│   └── src
│       ├── main
│       │   ├── resources
│       │   └── scala
│       │       ├── Branch1.scala
│       │       └── Branch2.scala
│       └── test
│           ├── resources
│           └── scala
├── ex03_sql_table_creation
│   ├── creation.sql
│   ├── data
│   │   └── taxi_zone_lookup.csv
│   ├── insertion.sql
│   ├── LocationDimInsertion.sql
│   └── peuplerdim_time.sql
├── ex04_dashboard
│   ├── EDA_nyc_taxi.ipynb
│   ├── requirement.txt
│   ├── streamlit_app.py
│   ├── utils
│   └── db_utils.py
├── ex05_ml_prediction_service
│   ├── artifacts
│   │   ├── metrics.json
│   │   ├── model.joblib
│   │   └── schema.json
│   ├── main.py
│   ├── pyproject.toml
│   ├── README.md
│   ├── requirements.txt
│   └── src
│       ├── config.py
│       ├── features.py
│       ├── io_minio.py
│       ├── model.py
│       ├── predict.py
│       ├── __pycache__
│       │   ├── config.cpython-311.pyc
│       │   ├── features.cpython-311.pyc
│       │   ├── io_minio.cpython-311.pyc
│       │   ├── model.cpython-311.pyc
│       │   ├── train.cpython-311.pyc
│       │   └── validate.cpython-311.pyc
│       ├── train.py
│       └── validate.py
│   ├── tests
│   │   ├── test_validate_infer_input.py
│   │   └── test_validate_train_input.py
│   └── uv.lock
├── ex06_airflow
│   └── README.md
├── README.md
├── target
│   ├── global-logging
│   └── task-temp-directory
├── temp.txt
├── yellow_tripdata_cleaned_2024-01.parquet
└── part-00000-2180301f-9fe9-4e35-8046-45348575936f-c000.snappy.parquet

```

Screenshot de l'arborescence du projet



Screenshot de l'architecture schématique

3. Développement technique

3.1 Mise en place de l'environnement

Nous avons utilisé Docker pour isoler les différents services :

- MinIO pour le stockage
- Airflow pour l'orchestration
- Services Python pour le traitement

Les commandes principales utilisées :

```
docker compose up -d
docker compose ps
```

Cette conteneurisation garantit la reproductibilité du projet.

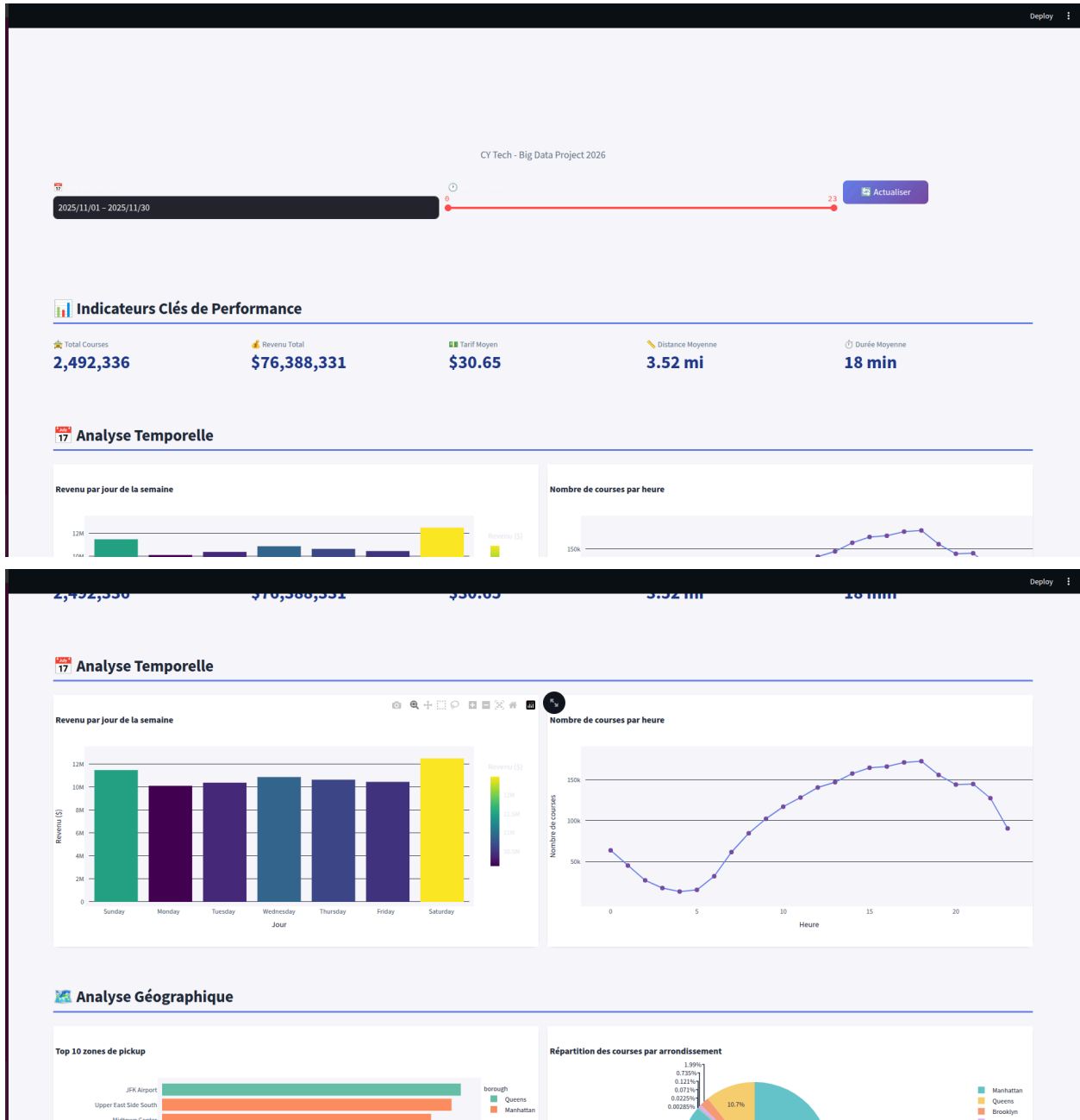
```
linam@linam-Vostro-3520:~/projet_big_data_cytech_25$ docker compose up -d
[+] Running 6/6
 ✓ Container minio                               Started
 ✓ Container projet_big_data_cytech_25-postgres-1 Started
 ✓ Container spark-master                         Started
 ✓ Container spark-worker-2                      Started
 ✓ Container spark-worker-1                      Started
 ✓ Container pgadmin                             Started
```

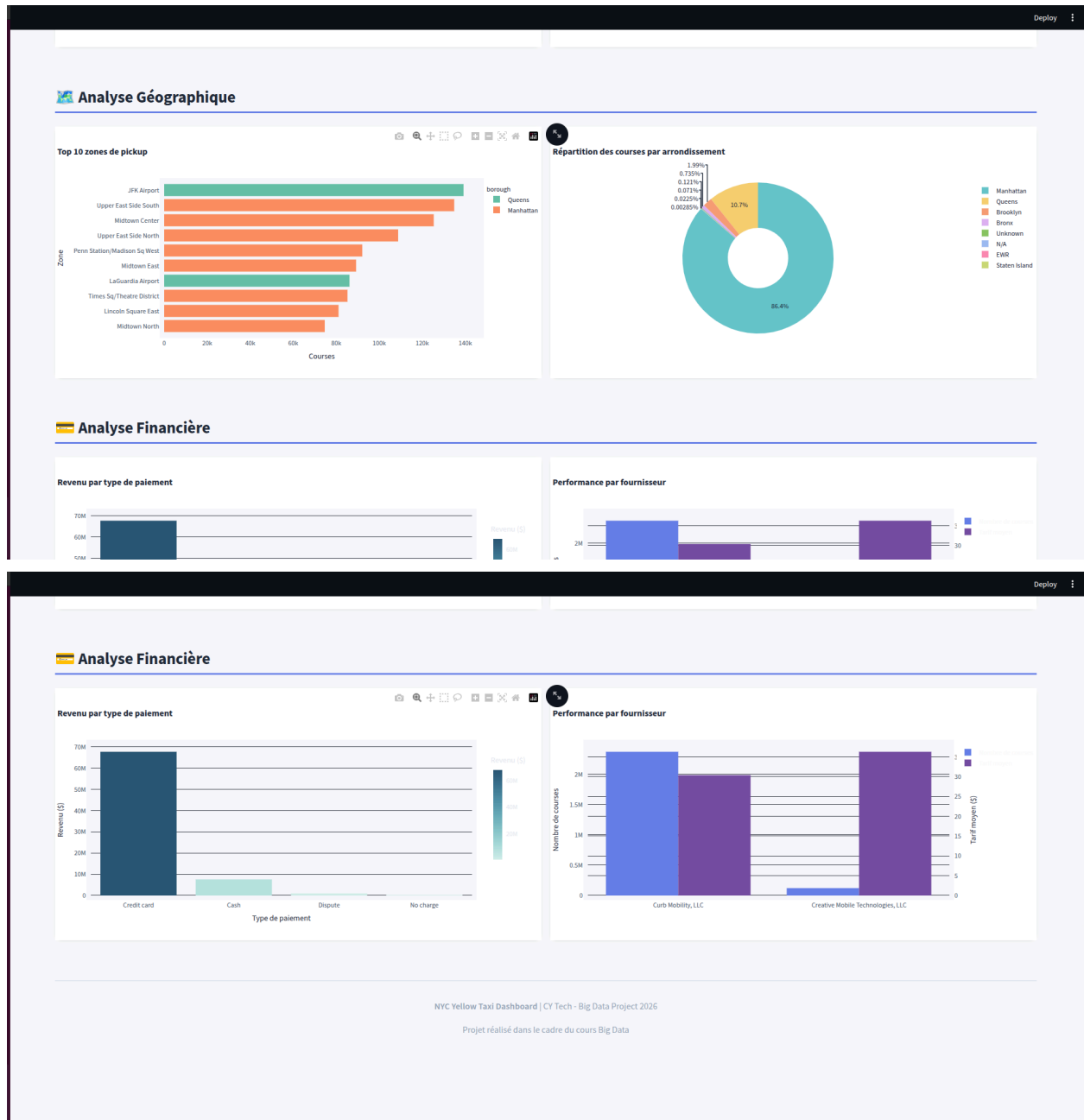
- Screenshot du lancement Docker

```
linam@linam-Vostro-3520:~/projet_big_data_cytech_25$ docker compose up -d
[+] Running 6/6
 ✓ Container minio                               Started           0.3s
 ✓ Container projet_big_data_cytech_25-postgres-1 Started           0.3s
 ✓ Container spark-master                         Started           0.3s
 ✓ Container spark-worker-2                      Started           0.7s
 ✓ Container spark-worker-1                      Started           0.7s
 ✓ Container pgadmin                             Started           0.7s
linam@linam-Vostro-3520:~/projet_big_data_cytech_25$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
02d58e41f444   dpape/pgadmin4:latest              "/entrypoint.sh"        10 days ago   Up 28 seconds   443/tcp, 0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
81fc2c0bedfc   pgadmin                             "docker-entrypoint.s..." 10 days ago   Up 29 seconds   0.0.0.0:5432->5432/tcp, :::5432->5432/tcp
cc9a5f1502db   projet_big_data_cytech_25-postgres-1 "/__cacert_entrypoin..." 11 days ago   Up 28 seconds   7000/tcp, 7077/tcp, 8080/tcp
ff2f806221e4   projet_big_data_cytech_25-spark-worker-2 "/__cacert_entrypoin..." 11 days ago   Up 28 seconds   7000/tcp, 7077/tcp, 8080/tcp
80b616454f93   projet_big_data_cytech_25-spark-worker-1 "/__cacert_entrypoin..." 11 days ago   Up 28 seconds   7000/tcp, 7077/tcp, 8080/tcp
8081->8080/tcp   projet_big_data_cytech_25-spark-master "/__cacert_entrypoin..." 11 days ago   Up 29 seconds (healthy) 7000/tcp, 0.0.0.0:7077->7077/tcp, :::7077->7077/tcp, 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
3dbe29983e6c   minio/minio:RELEASE.2025-09-07T16-13-09Z "/usr/bin/docker-ent..." 11 days ago   Up 29 seconds   0.0.0.0:9000-9001->9000-9001/tcp, :::9000-9001->9000-9001/tcp
linam@linam-Vostro-3520:~/projet_big_data_cytech_25$
```

- Screenshot de docker compose ps

3.2 Exercice 4 - Dashboard (screenshots)





3.3 Exercice 5 – Pipeline de Machine Learning

Objectif

Construire un pipeline complet permettant :

- Le chargement des données depuis MinIO

- Leur nettoyage
- L'entraînement d'un modèle
- L'évaluation des performances
- La génération de prédictions

Chargement des données

Nous avons implémenté une fonction permettant de charger un fichier Parquet depuis MinIO via l'API S3 (boto3).

Le fichier utilisé est :

yellow_tripdata_2025-05.parquet

Le dataset complet contenant plusieurs millions de lignes, nous avons volontairement limité le traitement à **10 000 lignes**. (en premier temps

Cette décision est justifiée par :

- Les limitations de mémoire RAM sur nos machines personnelles
- Le temps d'entraînement trop élevé sur l'ensemble du dataset
- L'objectif pédagogique centré sur la validation du pipeline plutôt que sur l'optimisation industrielle

```

/content/projet_big_data_cytech_25/ex05_ml_prediction_service# python -m src.train --data
/content/big_data_ml_project/yellow_tripdata_cleaned_2024-01.parquet --max-rows 0
Training progress: 0%|          | 0/7 [00:00<?, ?step/s]
2026-02-13 16:20:37,883 | INFO | ▶ Load parquet ...
2026-02-13 16:20:38,463 | INFO | Loaded dataframe: 2492831 rows, 20 cols
2026-02-13 16:20:38,463 | INFO | ✓Load parquet (0.58s)
Training progress: 14%|███      | 1/7 [00:00<00:03, 1.72step/s]
2026-02-13 16:20:38,464 | INFO | ▶ Validate input ...
2026-02-13 16:20:38,751 | INFO | ✓Validate input (0.29s)

```

Entrainement sur la totalité du data (yellow_tripdata for 01-2024)

Prétraitement

Le prétraitement comprend :

- Suppression des valeurs incohérentes
- Sélection des variables explicatives pertinentes
- Gestion des valeurs manquantes

Les variables utilisées incluent :

- trip_distance
- passenger_count
- fare_amount
- taxes

La variable cible est :

`total_amount`

```

1 {
2   "target": "total_amount",
3   "numeric_features": [
4     "trip_distance",
5     "trip_duration_min",
6     "passenger_count",
7     "pickup_hour",
8     "pickup_dayofweek",
9     "pickup_month"
10  ],
11  "categorical_features": [
12    "VendorID",
13    "RatecodeID",
14    "PULocationID",
15    "DOLocationID",
16    "store_and_fwd_flag",
17    "payment_type"
18  ],
19  "all_features": [
20    "trip_distance",
21    "trip_duration_min",
22    "passenger_count",
23    "pickup_hour",
24    "pickup_dayofweek",
25    "pickup_month",
26    "VendorID",
27    "RatecodeID",
28    "PULocationID",
29    "DOLocationID",
30    "store_and_fwd_flag",
31    "payment_type"
32  ]
33 }

```

- *Screenshot des colonnes sélectionnées*

Entraînement du modèle

Nous avons utilisé un modèle de régression (Random Forest).

L'évaluation a été réalisée avec le RMSE (Root Mean Squared Error).

Exemple de résultat obtenu :

RMSE : 6.52

Cela signifie que l'erreur moyenne sur la prédiction du montant total est d'environ 6.52 dollars.

```
{  
  "rmse": 6.525157038024782,  
  "n_train": 1994264,  
  "n_test": 498567  
}
```

- ***Screenshot du terminal affichant le RMSE***

Tests et qualité du code

Nous avons intégré :

- pytest pour les tests unitaires
- flake8 pour vérifier le style
- Documentation des fonctions au format NumpyDoc

Cette étape garantit la robustesse et la maintenabilité du code.

4. Répartition des tâches

Le travail a été réparti de manière équilibrée entre les membres du groupe.

Yanis Zidi

- Mise en place initiale de Docker
- Configuration des services MinIO
- Participation au développement du pipeline

Alexandre Bouharira

- Développement des fonctions de chargement et prétraitement
- Implémentation du modèle de Machine Learning
- Intégration des tests unitaires

Ilyes OubBea

- Structuration du projet et organisation Git
- Développement partiel des tests
- Assistance au débogage

Samuel Silva

- Validation des performances du modèle
- Analyse des résultats
- Contribution au prétraitement des données

LIMAM Mohamed Limam

- Gestion du dépôt GitHub
- Intégration finale des exercices
- Architecture du base de données pour l'analyse de data

Chaque membre a contribué aux phases de réflexion, aux tests et aux validations finales. Cette organisation a permis d'assurer une compréhension collective du projet.

5. Difficultés rencontrées

Nous avons rencontré plusieurs difficultés :

- Problèmes de configuration Docker
- Erreurs d'authentification MinIO
- Limitations de performance sur les datasets volumineux
- Conflits de versions de dépendances Python

Ces difficultés ont renforcé notre compréhension :

- De la gestion des environnements isolés
- De la manipulation des données volumineuses
- De l'importance des tests

6. Conclusion

Ce projet nous a permis de mettre en œuvre une architecture Big Data complète intégrant stockage, traitement, apprentissage automatique et orchestration.

Nous avons acquis une meilleure compréhension :

- Des pipelines de données
- De la structuration d'un projet complexe
- De l'importance de la reproductibilité
- Des contraintes matérielles dans le traitement de données volumineuses

L'intégration d'Airflow apporte une dimension plus professionnelle au projet en simulant un environnement proche des architectures industrielles modernes.

Ce projet constitue ainsi une synthèse concrète des compétences acquises dans le module Big Data.